Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

FINAL PROJECT REPORT

ON

NATURAL LANGUAGE INTERFACE FOR EDTECH INFRASTRUCTURE

SUBMITTED BY

SUNHA FATHIMA K P

REG NO :

23BCAICD103

GUIDED BY

MISS DEEKSHA SHETTY

SUBMISSION DATE

29-04-2026

Table of Contents

1 EXECUTIVE SUMMARY

2 BACKGROUND

2.1 AIM AND OBJECTIVES

2.1.1 Aim

2.1.2 Objectives

2.2 TECHNOLOGIES USED

2.2.1 Python

2.2.2Flask

2.2.3JSON (JavaScript Object Notation)

2.2.4HTML

2.2.5Logging Module

2.3 HARDWARE ARCHITECTURE

2.4 SOFTWARE ARCHITECTURE

2.4.1. User Interface Layer

2.4.2. Application Layer (Flask Backend)

2.4.3. Processing Layer (Chatbot Logic)

2.4.4. Data Layer

2.4.5. Logging Layer

3 SYSTEM OVERVIEW

3.1 Requirements

3.1.1 Functional Requirements

3.1.2 User Requirements

3.1.3 Environmental Requirements

3.2 Design and Architecture

3.2.1 System Design Approach

3.2.2 Data Flow Design

3.2.3 Design Considerations

3.2.4 Architectural Advantages

3.3 Implementation

3.3.1. Dataset Generation Module

3.3.2. Chatbot Logic Module

3.3.3. Flask Backend Module

3.3.4. User Interface Module

3.3.5. Logging and Error Handling

3.4 Testing

3.5 Graphical User Interface (GUI) Layout

3.6 Customer Testing

3.7 Evaluation

4 SNAPSHOTS OF THE PROJECT

4.1 Chatbot Interface

4.2 Sample User Interaction

4.3 Chatbot Response for Course Query

4.4 Multiple Interaction

4.5 Fallback Response

5 CONCLUSION

6 FURTHER DEVELOPMENT OR RESEARCH

7 REFERENCES

8 APPENDIX

8.1 Source Code Overview

8.2 Dataset Description

8.3 Sample Execution Logs

8.4 System Execution Steps

CHAPTER 1

EXECUTIVE SUMMARY

In today's digital age, students often face difficulty in finding reliable and organized information about colleges, courses, fees, placements, and related facilities. Searching through multiple sources can be time-consuming and confusing. To address this issue, an AI-based EdTech Chatbot is developed to provide quick and accurate educational guidance.

The main objective of this project is to design a chatbot system that can interact with users and respond to their queries related to colleges and courses. The system is built using a rule-based Natural Language Processing (NLP) approach, where user inputs are analyzed using predefined logic and matched with relevant responses.

The project is implemented using Python and Flask. The chatbot logic is developed in Python, while Flask is used to create the web application interface. A JSON dataset is used to store information about colleges, courses, fees, placements, and hostel facilities.

The system follows a simple architecture consisting of user input, processing, decision-making, and response generation. When a user enters a query, the system processes the input, identifies keywords, and provides the most appropriate response based on the dataset.

The chatbot is capable of handling different types of queries such as college recommendations, course details, fee structure, placement opportunities, and hostel information. It also includes basic features like typo correction and context handling to improve user interaction.

Overall, the system provides a simple, fast, and user-friendly solution for accessing educational information. Although it is rule-based and has certain limitations, it demonstrates how basic AI concepts can be applied to build practical applications. The project can be further enhanced by integrating advanced machine learning techniques and real-time data sources.

CHAPTER 2

BACKGROUND

The rapid growth of digital technology has transformed the education sector, making a vast amount of information available online. Students today have access to numerous sources for exploring colleges, courses, and career opportunities. However, this abundance of information often leads to confusion, as it is not always organized or easy to understand.

Many students struggle to find accurate details about colleges, fee structures, placement opportunities, and facilities like hostels. Traditional methods of searching through websites or consulting multiple sources are time-consuming and inefficient. This highlights the need for a smart and interactive system that can provide quick and reliable educational guidance.

Chatbots have emerged as an effective solution for such problems. They are capable of simulating human-like conversations and providing instant responses to user queries. In the education domain, chatbots can simplify the process of information retrieval by acting as a virtual assistant.

This project focuses on developing an **AI-based EdTech Chatbot** that helps students access relevant educational information in a simple and efficient manner. The system is designed using a rule-based approach, where user inputs are analyzed and matched with predefined responses to generate meaningful outputs.

2.1 AIM AND OBJECTIVES

2.1.1 Aim

The aim of this project is to develop an AI-based EdTech Chatbot that can assist students by providing accurate information about colleges, courses, fees, placements, and hostel facilities through an interactive and user-friendly interface.

2.1.2 Objectives

- To design and develop a chatbot system that can understand user queries and provide relevant responses.
- To provide detailed information about various types of colleges such as engineering, arts, medical, and dental institutions.
- To display course-related details including duration, fees, and availability.
- To provide information regarding placement opportunities and hostel facilities in different colleges.

- To implement a keyword-based Natural Language Processing (NLP) approach for processing user input.
- To develop a web-based interface using Flask for smooth user interaction.
- To ensure quick and accurate response generation for different types of queries.
- To create and manage a structured dataset for storing educational information.

2.2 TECHNOLOGIES USED

The development of the EdTech Chatbot system involves the use of the following technologies:

2.2.1 Python

Python is used as the primary programming language for developing the chatbot. It is chosen due to its simplicity, readability, and strong support for libraries related to data processing and web development.

2.2.2Flask

Flask is a lightweight web framework used to build the backend of the application. It handles user requests, processes input data, and connects the frontend interface with the chatbot logic.

2.2.3JSON (JavaScript Object Notation)

JSON is used to store structured data related to colleges, courses, fees, placement details, and hostel information. It allows efficient data storage and easy retrieval within the system.

2.2.4HTML

HTML is used to design the frontend interface of the chatbot. It provides the basic structure of the web page where users interact with the system.

2.2.5Logging Module

The logging module in Python is used to record user queries and chatbot responses. This helps in debugging and monitoring the system performance.

2.3 HARDWARE ARCHITECTURE

The hardware architecture of the EdTech Chatbot system is simple and does not require any specialized equipment. The application is designed to run on a standard personal computer.

The system follows a basic client-server approach, where the user interacts with the chatbot through a web browser, and the backend processing is handled by the system running the Flask server.

Hardware Requirements:

- **Processor:** Minimum Intel i3 or equivalent
- **RAM:** 4 GB or above
- **Storage:** At least 500 MB free space
- **System Type:** 64-bit operating system
- **Input Devices:** Keyboard and mouse
- **Output Devices:** Monitor

Since the system is lightweight, it can run efficiently on basic hardware without any performance issues.

2.4 SOFTWARE ARCHITECTURE

The EdTech Chatbot system follows a modular software architecture, where each component is designed to perform a specific function. This approach improves the clarity, maintainability, and scalability of the system.

The architecture is divided into the following layers:

2.4.1. User Interface Layer

This layer provides the interface through which users interact with the chatbot. It is developed using HTML and displayed in a web browser. Users can enter their queries in a text input field and receive responses in the chat window.

2.4.2. Application Layer (Flask Backend)

This layer acts as the bridge between the user interface and the chatbot logic. It is implemented using Flask, a lightweight web framework. It handles HTTP requests, processes user input, and sends responses back to the frontend through an API endpoint.

2.4.3. Processing Layer (Chatbot Logic)

In this layer, the core chatbot functionality is implemented using Python. The user input is processed through:

- Text normalization
- Keyword detection
- Conditional logic for response generation

The system uses a rule-based approach to match user queries with predefined responses.

2.4.4. Data Layer

This layer manages the storage and retrieval of data. A JSON file is used to store information about colleges, courses, fees, placement details, and hostel facilities. The chatbot accesses this data to generate accurate responses.

2.4.5. Logging Layer

The system includes a logging mechanism that records user queries and chatbot responses. This helps in monitoring system performance and debugging errors.

The overall workflow of the system is as follows:
User input → Flask server → Chatbot processing → Data retrieval → Response generation → Output to user.

This modular architecture ensures that each component functions independently while contributing to the overall system performance. It also makes the system easy to update and extend in the future.

CHAPTER 3

SYSTEM OVERVIEW

The EdTech Chatbot system is developed to provide students with quick and reliable information related to colleges, courses, fees, placements, and hostel facilities through an interactive chat interface. The system acts as a virtual assistant that responds to user queries in real time, reducing the need for manual searching across multiple sources.

The chatbot is based on a rule-based approach, where user inputs are analyzed and matched with predefined conditions to generate appropriate responses. It is capable of handling different types of queries such as finding the best colleges, listing available courses, and providing detailed information about fees, placements, and accommodation.

The working of the system begins when a user enters a query through the web interface. The input is sent to the Flask backend, where it is processed using the chatbot logic. The system performs text normalization and keyword detection to understand the user's intent. Based on this, relevant data is retrieved from the dataset, and a suitable response is generated and displayed to the user.

The system is designed with simplicity and efficiency in mind. It uses a structured JSON dataset to ensure consistency and accuracy in responses. The modular design allows different components such as the user interface, backend server, chatbot logic, and data storage to function independently while working together as a complete system.

One of the key features of the system is its ability to handle multiple query types within a single interface. It also includes basic enhancements such as typo correction and context handling, which improve the overall user experience.

Although the chatbot is limited by its rule-based nature and may not fully understand complex natural language queries, it effectively demonstrates how basic AI techniques can be applied to build a practical and useful application. The system can be further enhanced by integrating advanced Natural Language Processing (NLP) methods and real-time data sources.

3.1 Requirements

The development of the EdTech Chatbot system is based on a set of requirements that define how the system should function and what it should deliver to the users. These requirements help in ensuring that the system performs efficiently, provides accurate information, and maintains a smooth user experience.

The requirements are categorized into functional requirements, user requirements, and environmental requirements. Each category focuses on a specific aspect of the system to ensure its proper operation and usability.

3.1.1 Functional Requirements

Functional requirements describe the core operations that the system must perform. These requirements ensure that the chatbot provides the expected features and services.

- The system should accept user input through a web-based interface.
- The system should process user queries and identify relevant keywords.
- The system should provide information about different types of colleges such as engineering, arts, medical, and dental.
- The system should display course details including duration and fees.
- The system should provide information about placement opportunities.
- The system should display hostel and accommodation details when requested.
- The system should recommend top colleges based on user input such as category and location.
- The system should handle multiple types of queries within a single interface.
- The system should provide appropriate responses for invalid or unknown inputs.
- The system should ensure smooth communication between frontend and backend.

These functional requirements ensure that the chatbot performs all necessary tasks effectively.

3.1.2 User Requirements

User requirements define the expectations of the end users who interact with the system. The chatbot is mainly designed for students seeking educational information.

- The system should provide a simple and user-friendly interface.
- The system should allow users to enter queries easily without technical knowledge.
- The chatbot should provide quick and clear responses.
- The information provided should be accurate and easy to understand.
- The system should support different types of queries in a single platform.

- The interface should be accessible through a web browser.
- The system should help users find information about colleges, courses, fees, placements, and hostel facilities.

These requirements ensure that the system is convenient and effective for users.

3.1.3 Environmental Requirements

Environmental requirements specify the conditions and setup needed for the system to function properly.

- The system should run in a Python-supported environment.
- A web browser such as Chrome or Edge is required to access the interface.
- The system should support operating systems like Windows, Linux, or macOS.
- Required software such as Python and necessary libraries must be installed.
- A stable system environment is required for running the Flask server smoothly.

These requirements ensure that the system operates efficiently without the need for specialized hardware or software.

3.2 Design and Architecture

The design of the EdTech Chatbot system focuses on simplicity, modularity, and efficient processing of user queries. The system is structured in such a way that each component performs a specific task, ensuring smooth interaction between different modules.

The architecture follows a modular approach, where the system is divided into multiple layers that work together to process user input and generate appropriate responses. This design makes the system easy to understand, maintain, and extend in the future.

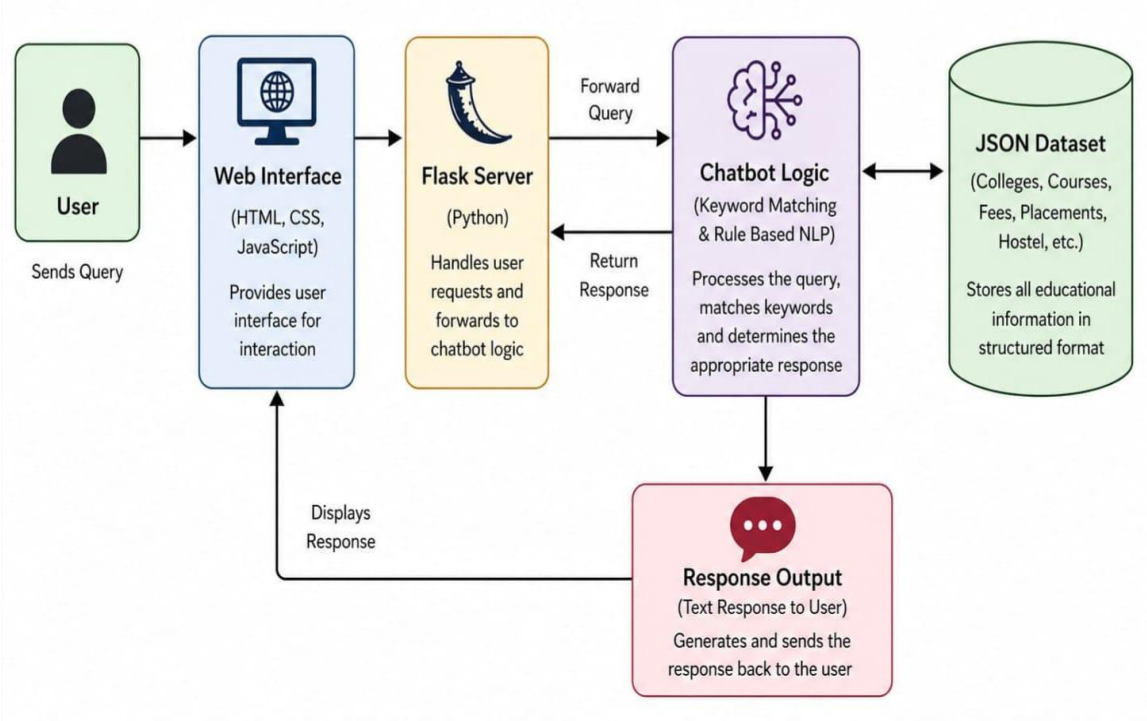


Figure 3.1 System Architecture of Edtech chatbot

3.2.1 System Design Approach

The EdTech Chatbot system is designed using a modular and rule-based approach to ensure simplicity, clarity, and efficient performance. The design focuses on breaking the system into smaller components, where each module performs a specific function. This makes the system easy to develop, maintain, and extend.

The chatbot follows a rule-based Natural Language Processing (NLP) approach. In this method, user input is processed by identifying keywords and matching them with predefined conditions to generate appropriate responses. This approach is suitable for handling structured queries related to colleges, courses, fees, placements, and hostel facilities.

The system is designed using a client-server model. The user interacts with the chatbot through a web interface (client side), while the backend processing is handled by a Flask server. The server receives user input, processes it using the chatbot logic, and sends back the response.

The design also emphasizes simplicity and usability. The user interface is kept minimal so that users can easily interact with the system without requiring technical knowledge. At the same time, the backend is structured in a way that ensures fast processing and accurate response generation.

Additionally, the system uses a structured JSON dataset to store all required information. This allows quick data retrieval and ensures consistency in responses.

Overall, the chosen design approach ensures that the system is efficient, easy to use, and suitable for the intended purpose, while also providing flexibility for future improvements such as integrating advanced NLP techniques.

3.2.2 Data Flow Design

The data flow design of the EdTech Chatbot system describes how information moves through different components of the system, from user input to response generation. It ensures that data is processed efficiently and the output is delivered to the user in real time.

The process begins when the user enters a query through the web interface. This input is sent to the Flask backend using an HTTP request. The backend receives the input and forwards it to the chatbot processing module.

In the processing stage, the system performs text normalization and keyword detection to understand the intent of the user query. Based on the identified keywords, the chatbot applies predefined logic to determine the appropriate response.

The system then retrieves relevant information from the JSON dataset, which contains details about colleges, courses, fees, placements, and hostel facilities. The selected data is used to generate a response.

Finally, the response is sent back to the Flask backend, which returns it to the frontend interface. The output is then displayed to the user in the chat window.

The overall data flow can be summarized as follows:

User Input → Flask Backend → Chatbot Processing → Data Retrieval (JSON) → Response Generation → Output Display

This structured flow ensures that the system processes user queries efficiently and provides accurate responses without delay.

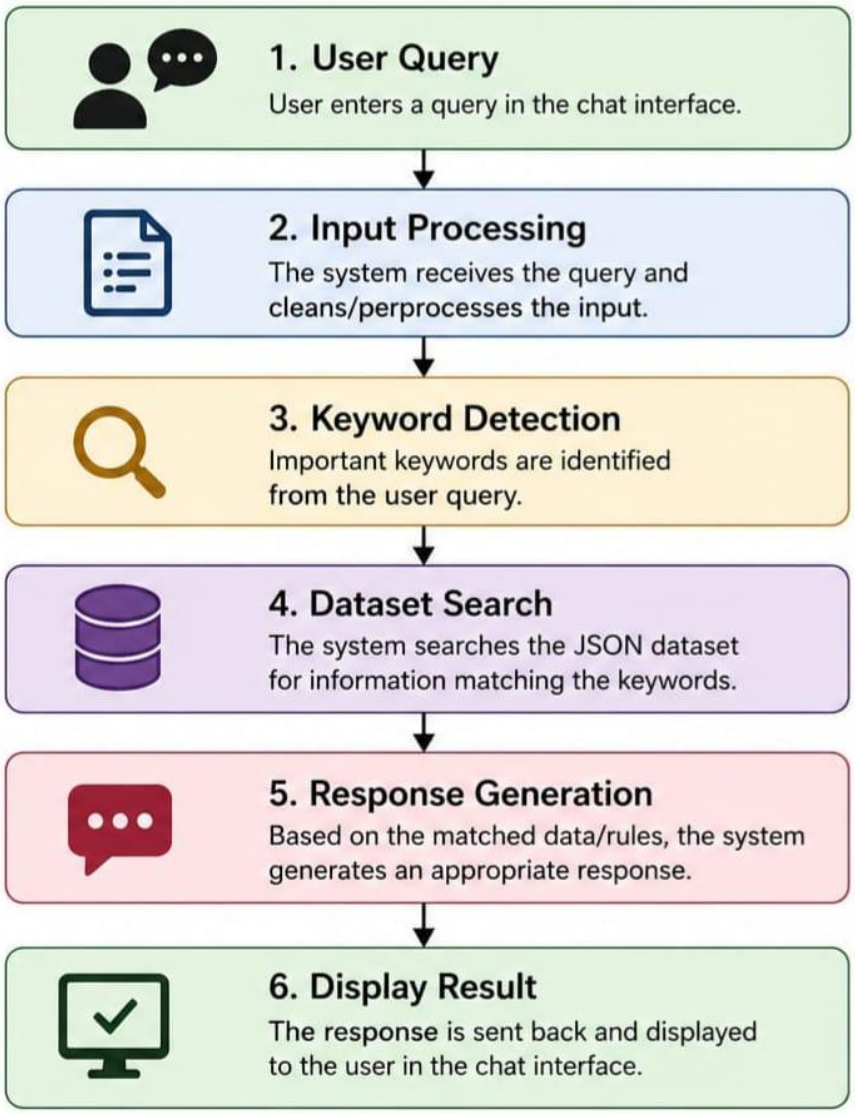


Figure 3.2 Data Flow Design

3.2.3 Design Considerations

The design of the EdTech Chatbot system is guided by key considerations to ensure usability, efficiency, and reliability.

- **Simplicity and Usability**
  The interface is kept minimal and easy to use so that students can interact with the chatbot without technical knowledge.

- **Modular Design**
  The system is divided into independent modules (frontend, backend, chatbot logic, and data layer), making it easier to develop, test, and maintain.

- **Performance and Responsiveness**
  The system is designed to provide quick responses by using lightweight processing and efficient data access from the JSON dataset.
- **Data Organization**
  Information is stored in a structured JSON format, allowing easy retrieval and consistent response generation.
- **Scalability**
  The architecture allows future enhancements such as adding more data, improving logic, or integrating advanced NLP techniques.
- **Error Handling and Reliability**
  The system includes basic validation and fallback responses to handle invalid or unknown inputs, ensuring stable operation.
- **Maintainability**
  Clear code structure and separation of components make it easier to update and modify the system.

3.2.4 Architectural Advantages

The architecture of the EdTech Chatbot system offers several advantages that improve its performance, usability, and maintainability.

- **Modular Structure**
  The system is divided into independent modules such as frontend, backend, chatbot logic, and data storage. This makes development, testing, and maintenance easier.
- **Ease of Use**
  The simple web-based interface allows users to interact with the system without any technical knowledge.
- **Efficient Processing**
  The rule-based approach ensures fast response generation, as the system directly matches user queries with predefined conditions.
- **Flexibility**
  The system can be easily updated by modifying the dataset or improving the chatbot logic without affecting other components.
- **Scalability**
  The architecture supports future enhancements such as adding more features, expanding the dataset, or integrating advanced NLP techniques.

- **Low Resource Requirement**
  The system is lightweight and can run on basic hardware without requiring high computational power.
- **Reliability**
  The use of structured data and predefined logic ensures consistent and predictable responses.

3.3 Implementation

The implementation of the EdTech Chatbot system is carried out using Python and Flask. The system is developed in a modular manner, where each component is responsible for a specific function. This approach improves clarity, simplifies debugging, and makes the system easier to maintain and extend.

The implementation is divided into the following modules:

3.3.1. Dataset Generation Module

The dataset is created using a Python script. It contains structured information about colleges, courses, fees, placement opportunities, and hostel facilities. The data is stored in JSON format, which allows easy access and efficient data handling.

The dataset includes different categories such as engineering, arts, medical, and dental colleges. Each college contains multiple courses along with relevant details. In addition, short-term courses with duration, fees, and placement information are also included.

3.3.2. Chatbot Logic Module

The chatbot logic is implemented in Python and forms the core of the system. This module is responsible for processing user input and generating appropriate responses.

The implementation includes:

- Text normalization to handle minor typing errors
- Keyword detection to identify user intent
- Conditional logic to generate responses

The chatbot is capable of handling queries related to:

- College recommendations
- Course details
- Fee structure
- Placement opportunities

- Hostel information

It also maintains basic context using previously selected course details, which helps in improving response accuracy.

3.3.3. Flask Backend Module

The backend is developed using Flask, which acts as an interface between the frontend and the chatbot logic. It handles user requests and ensures smooth communication within the system.

The backend performs the following functions:

- Receives user input through an API endpoint
- Sends the input to the chatbot module
- Returns the generated response to the user

This enables real-time interaction between the user and the chatbot.

3.3.4. User Interface Module

The user interface is designed using HTML and provides a simple chat-based interaction platform. Users can enter their queries through a text input field and receive responses in the chat window.

The interface includes:

- Input text field
- Chat display area
- Send button

The design is kept minimal to ensure ease of use.

3.3.5. Logging and Error Handling

The system includes a logging mechanism that records user queries and chatbot responses. This helps in monitoring system performance and debugging issues.

Basic error handling is also implemented to manage invalid inputs and unexpected errors, ensuring stability and smooth functioning of the system.

Overall, the implementation ensures that all modules work together efficiently to process user queries and provide accurate responses. The modular design also allows future improvements and feature enhancements.

3.4 Testing

Testing is an essential phase in the development of the EdTech Chatbot system. It is performed to verify that the system functions correctly, provides accurate responses, and delivers a smooth user experience. The chatbot was tested using different types of inputs and scenarios to evaluate its performance, reliability, and correctness.

Types of Testing Performed

- **Functional Testing**
  Functional testing was conducted to ensure that all core features of the system are working as expected. This includes verifying functionalities such as college recommendations, course details, fee information, placement details, and hostel facilities.
- **Input Testing**
  The system was tested with various types of inputs, including valid queries, invalid inputs, incomplete sentences, and random text. The chatbot was able to handle most inputs effectively and provide appropriate responses or fallback messages.
- **Performance Testing**
  The response time of the chatbot was evaluated to ensure that it provides quick replies. The system responded almost instantly in most cases without noticeable delay.
- **Integration Testing**
  Integration testing was performed to ensure proper communication between the frontend interface, Flask backend, chatbot logic, and dataset. The system successfully processed user inputs and returned responses without errors.

Sample Test Cases

Test Case	Input	Expected Output	Result
1	Best engineering colleges in Kochi	List of top colleges	Passed
2	BCA course details	Display course information	Passed
3	Fees for Python course	Show fee details	Passed
4	Hostel facility	Provide hostel information	Passed
5	Random or unclear input	Default response message	Passed

Test Results

- The chatbot successfully handled most keyword-based queries.

- The responses were accurate and relevant to user inputs.
- Invalid or unclear inputs were handled using fallback responses.
- The system maintained stable performance during continuous usage.
- All modules worked together without integration issues.

Overall, the testing confirms that the EdTech Chatbot system performs effectively and meets its functional requirements. However, improvements can be made in handling complex or ambiguous queries by incorporating advanced Natural Language Processing techniques.

3.5 Graphical User Interface (GUI) Layout

The EdTech Chatbot system uses a simple web-based graphical user interface designed to provide an easy and interactive user experience. The interface is developed using HTML and is integrated with the Flask backend for real-time communication.

The GUI follows a chat-based layout where users can interact with the system by entering their queries and viewing responses in the same window. The design is kept minimal to ensure that users can navigate and use the system without any technical difficulty.

The main components of the interface include:

- **Chat Display Area**
  This section displays the conversation between the user and the chatbot. Both user inputs and system responses are shown in a clear and readable format.
- **Input Text Box**
  Users can type their queries in this field. It allows easy input of questions related to colleges, courses, fees, placements, and hostel facilities.
- **Send Button**
  The send button is used to submit the user's query to the system. Once clicked, the input is processed and the response is displayed instantly.

The communication between the frontend and backend is handled through a Flask API, which processes user input and returns the appropriate response. This ensures smooth and real-time interaction.

Overall, the GUI is designed with a focus on simplicity, clarity, and ease of use, enabling users to quickly access educational information without any complexity.

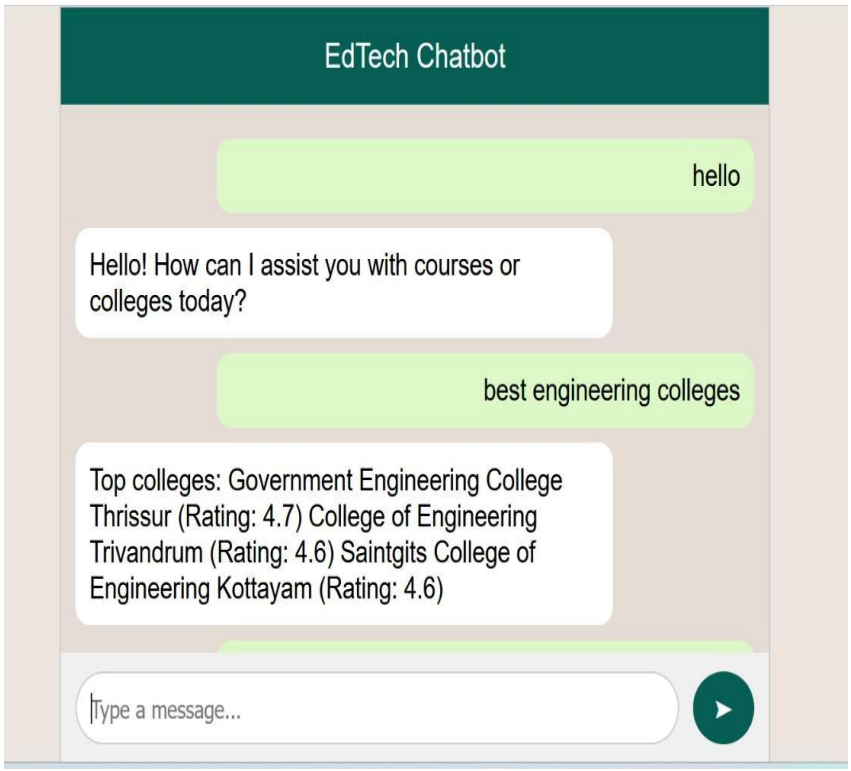


Figure 3.3: Sample interaction showing user query and chatbot response

3.6 Customer Testing

Customer testing was conducted to evaluate the usability and effectiveness of the EdTech Chatbot system. The system was tested by a small group of users, mainly students, to understand how well it performs in real-world usage.

The users were asked to interact with the chatbot by providing different types of queries related to colleges, courses, fees, placements, and hostel facilities. They were encouraged to test the system using both simple and slightly varied inputs.

Feedback from Users

- The chatbot was easy to use and understand.
- The interface was simple and user-friendly.
- The system provided quick responses to queries.
- The information provided was helpful for basic guidance.
- Users were able to interact with the system without any technical difficulty.

Observations

- The chatbot performed well for keyword-based queries.

- It successfully handled common questions related to educational information.
- Some limitations were observed in handling complex or unclear queries.

Overall, customer testing indicates that the system is effective for providing basic educational information and offers a smooth user experience. However, improvements can be made in enhancing the chatbot's ability to understand more complex queries.

3.7 Evaluation

The EdTech Chatbot system was evaluated based on its performance, usability, and ability to meet the project objectives. The evaluation focuses on identifying the strengths of the system as well as its limitations.

Performance Evaluation

The system provides fast responses to user queries due to its rule-based design. Since the chatbot directly matches user input with predefined conditions, the response time is minimal. The system performs efficiently on basic hardware without any noticeable delay.

Usability Evaluation

The chatbot offers a simple and user-friendly interface that allows users to interact easily without requiring technical knowledge. The chat-based interaction makes it intuitive for users to ask questions and receive answers quickly.

Functional Evaluation

The system successfully meets its main objectives by providing information about colleges, courses, fees, placements, and hostel facilities. It handles different types of queries within a single interface and generates relevant responses based on the dataset.

Advantages

- Simple and easy to use
- Fast response time
- Lightweight and efficient
- Provides structured and relevant information
- Easy to maintain and update

Limitations

- The system is rule-based and does not fully understand complex natural language queries
- It depends on predefined keywords and conditions
- The accuracy of responses depends on the dataset quality
- Limited ability to handle ambiguous or unexpected inputs

Overall, the system performs effectively for its intended purpose and demonstrates how basic AI techniques can be applied to create a useful application. However, future improvements can be made by integrating advanced Natural Language Processing techniques to enhance the chatbot's intelligence and flexibility.

CHAPTER 4

SNAPSHOTS OF THE PROJECT

This chapter presents the visual representation of the EdTech Chatbot system through screenshots. These snapshots demonstrate the working of the system and provide a clear understanding of the user interface and functionality.

4.1 Chatbot Interface

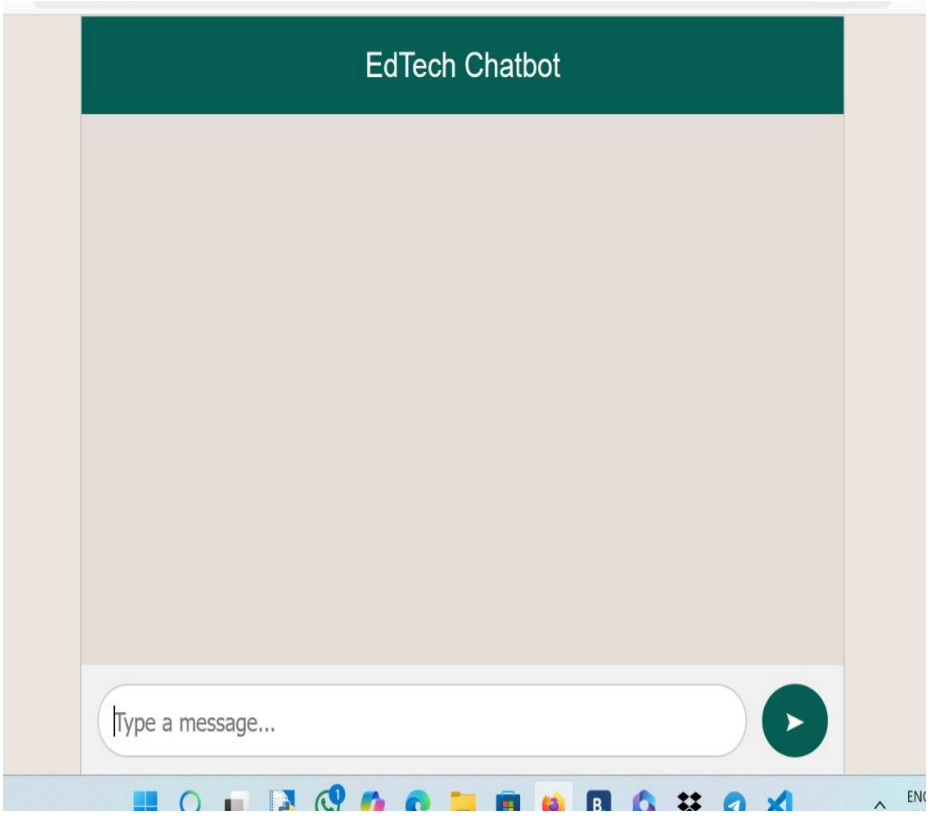


Figure 4.1: Chatbot User Interface

The above figure shows the main interface of the chatbot system. The user can enter queries in the input field and receive responses in the chat window. The interface is designed to be simple and user-friendly.

4.2 Sample User Interaction

This figure shows a sample interaction between the user and the chatbot. The user enters a query related to colleges or courses, and the chatbot provides a relevant response based on the dataset.

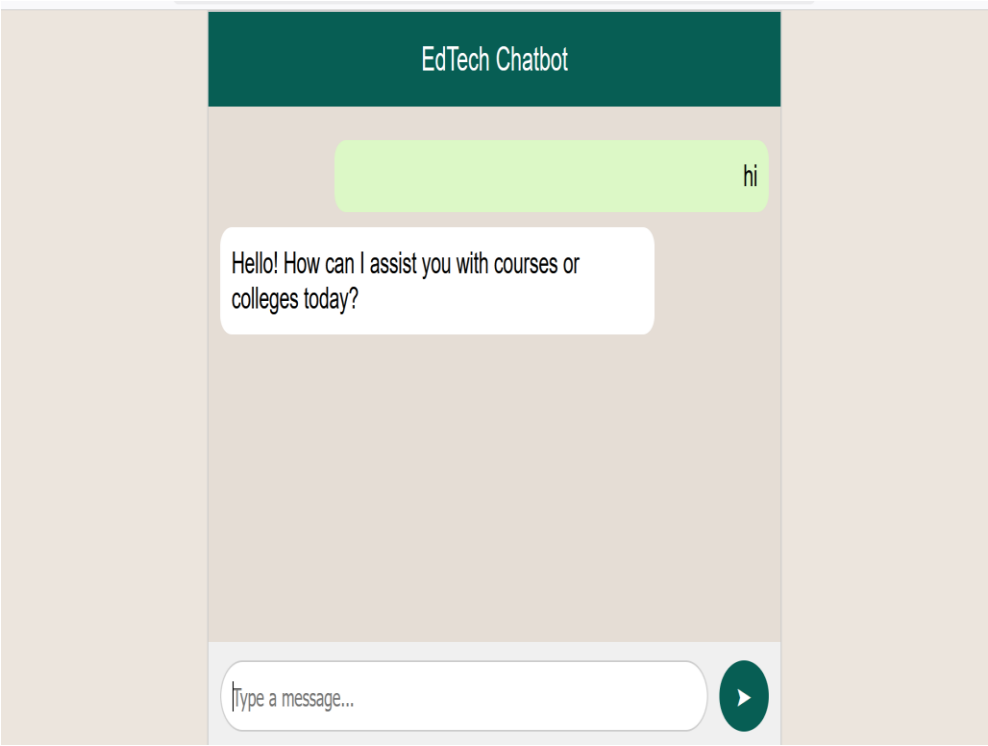


Figure 4.2: Sample Interaction between User and Chatbot

4.3 Chatbot Response for Course Query

This figure demonstrates how the chatbot provides course-related information such as duration, fees, and placement details based on the user's query.

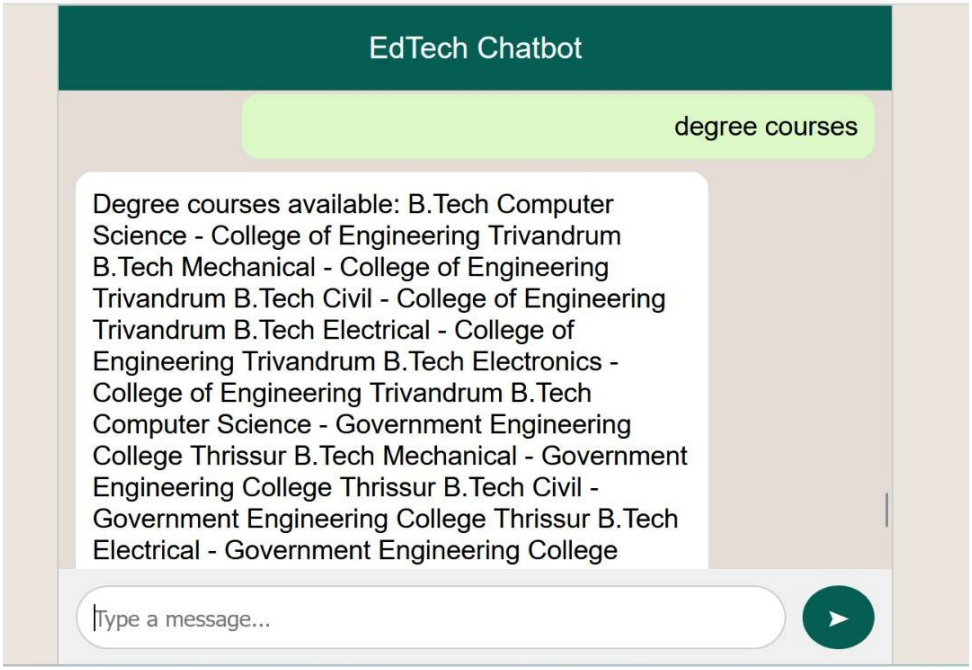
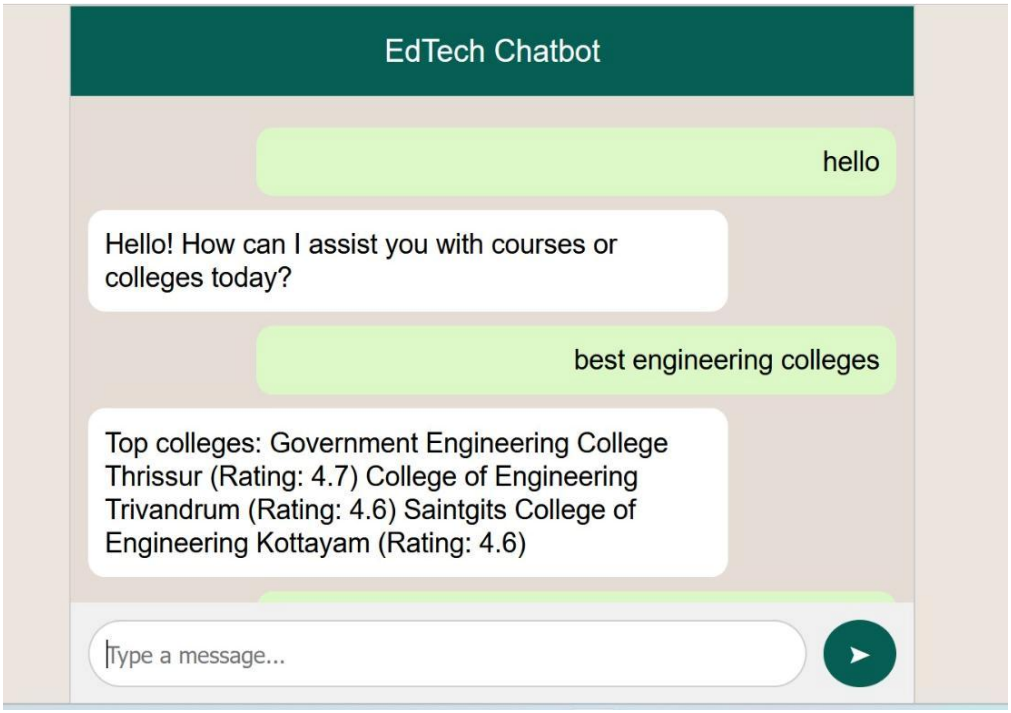


Figure 4.3: Chatbot Response for Course Query

4.4 Multiple Interaction



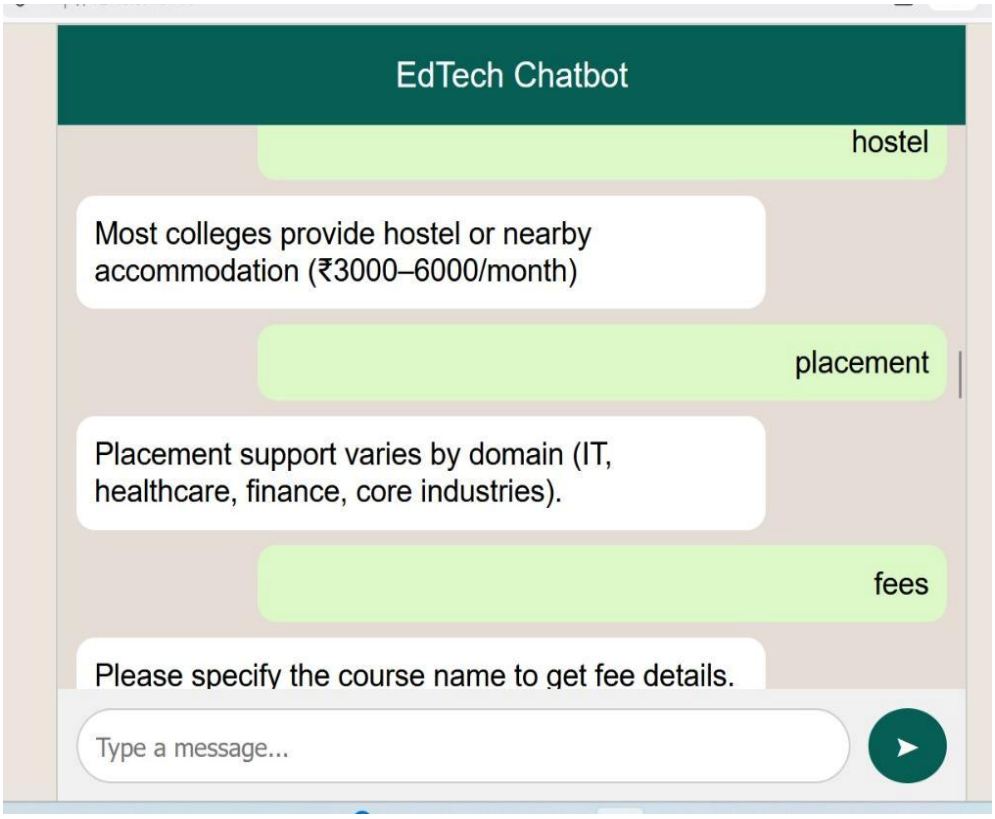


Figure 4.4: Multiple Interaction

This figure shows continuous interaction between the user and the chatbot, demonstrating conversational flow.

4.5 Fallback Response

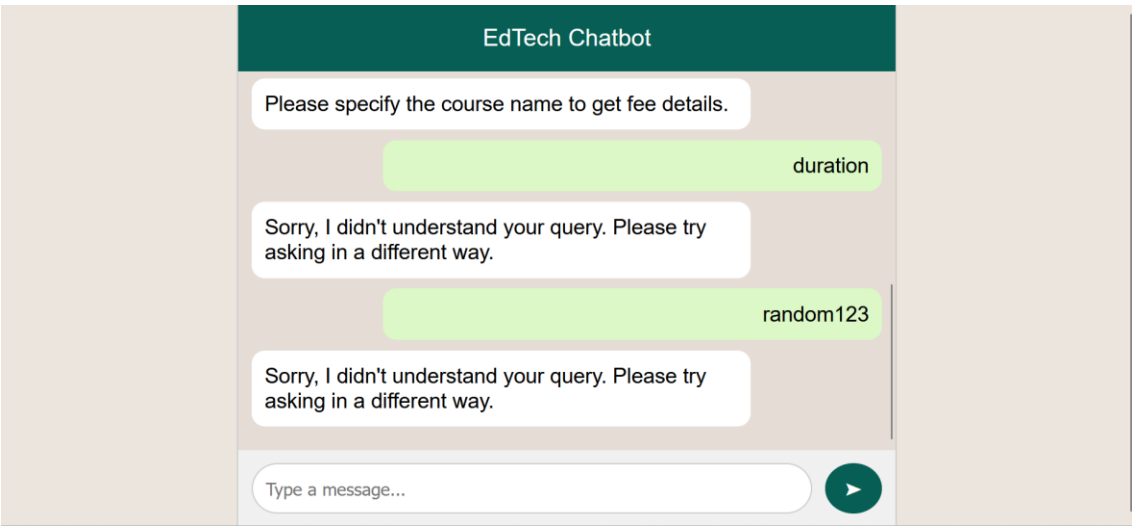


Figure 4.5: Fallback Response

When the chatbot cannot understand a query, it provides a default response instead of giving incorrect information.

CHAPTER 5

CONCLUSION

The EdTech Chatbot system developed in this project provides a simple and effective solution for accessing educational information. The system is designed to assist students by offering details about colleges, courses, fees, placements, and hostel facilities through an interactive chat interface.

The project successfully meets its objectives by implementing a rule-based chatbot that can process user queries and generate relevant responses. The use of Python and Flask enables smooth interaction between the frontend and backend, while the JSON dataset ensures structured and consistent data handling.

The system is easy to use, lightweight, and capable of providing quick responses. It reduces the effort required for manual searching and makes information more accessible to users. The modular design of the system also makes it easy to maintain and extend.

However, the chatbot has certain limitations due to its rule-based approach. It may not handle complex or ambiguous queries effectively and relies on predefined logic for response generation.

Overall, the project demonstrates how basic artificial intelligence concepts can be applied to build a practical and useful application. It also provides a strong foundation for further improvements and enhancements in the future.

CHAPTER 6

FURTHER DEVELOPMENT OR RESEARCH

The current EdTech Chatbot system provides a basic and effective solution for retrieving educational information. However, there are several areas where the system can be further improved and enhanced to increase its functionality and performance.

One of the major improvements can be the integration of advanced Natural Language Processing (NLP) techniques. Instead of a rule-based approach, machine learning or deep learning models can be used to better understand user queries and provide more accurate and context-aware responses.

The system can also be enhanced by connecting it to real-time databases or online APIs. This would allow the chatbot to provide updated information about colleges, courses, fees, and placement opportunities instead of relying on a static dataset.

Another area of improvement is the user interface. The current interface is simple, but it can be made more interactive and visually appealing by adding features such as chat bubbles, typing indicators, and better layout design.

Voice-based interaction can also be introduced to make the system more accessible. Users could interact with the chatbot using speech instead of typing, which would improve usability, especially for mobile users.

The chatbot can be further expanded by adding more features such as career guidance, entrance exam information, and personalized recommendations based on user preferences.

In addition, the system can be deployed on cloud platforms to make it accessible to a larger number of users. Mobile application support can also be developed to increase accessibility.

Overall, these improvements can transform the chatbot from a basic informational tool into a more intelligent and interactive educational assistant.

CHAPTER 7

REFERENCES

The following references were used during the development of the EdTech Chatbot system:

1. Python Software Foundation, *Python Documentation*. Available at: https://docs.python.org/
2. Pallets Projects, *Flask Documentation*. Available at: https://flask.palletsprojects.com/
3. JSON Official Website, *Introducing JSON*. Available at: https://www.json.org/
4. W3Schools, *HTML Tutorial*. Available at: https://www.w3schools.com/html/
5. W3Schools, *Python Tutorial*. Available at: https://www.w3schools.com/python/
6. TutorialsPoint, *Flask Tutorial*. Available at: https://www.tutorialspoint.com/flask/
7. GeeksforGeeks, *Chatbot Development in Python*. Available at: https://www.geeksforgeeks.org/

CHAPTER 8

APPENDIX

This chapter provides supporting details of the EdTech Chatbot system, including source code structure, dataset information, and sample execution logs. The complete source code is maintained separately and is referenced here for understanding the system implementation.

8.1 Source Code Overview

The system is implemented using multiple Python and frontend files, each handling a specific functionality:

- **app.py**
  This file contains the Flask application that handles routing and communication between the frontend and backend. It receives user input through an API and returns chatbot responses

```
from flask import Flask, render_template, request, jsonify

from chatbot import get_response


app = Flask(__name__) @app.route("/")

def home():

 return render_template("index.html")


@app.route("/chat", methods=["POST"])

def chat():

   try:

      user_input = request.json.get("message", "").strip()

      if not user_input:

         return jsonify({"response": "Please enter a message."})

         response = get_response(user_input)

         return jsonify({"response": response})

  except Exception as e:
```

```
    return jsonify({"response": "Something went wrong. Please try again."})
if __name__ == "__main__":
    print("Server running at: http://127.0.0.1:5000/")
app.run(debug=True, use_reloader=False)
```

- **chatbot.py**
  This file contains the core chatbot logic. It processes user input using keyword-based rules, retrieves data from the dataset, and generates appropriate responses

```
import json
import logging

# Load dataset
with open("data/dataset.json", encoding="utf-8") as f:
    data = json.load(f)

# Logging setup
logging.basicConfig(
    filename="logs/chatbot.log",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)

last_course = None

def normalize_text(text):
    """Basic typo correction"""
    corrections = {
        "colle": "college",
        "engg": "engineering",
```

```
    "med": "medical"
    }
    for wrong, correct in corrections.items():
        text = text.replace(wrong, correct)
    return text

def get_response(user_input):
    global last_course

    text = normalize_text(user_input.lower().strip())

    # ------------------ Greeting ------------------
    if text in ["hi", "hello", "hey"]:
        response = "Hello! How can I assist you with courses or colleges today?"

    # ------------------ BEST COLLEGE (ALL TYPES) ------------------
    elif "best" in text:

        categories = ["engineering", "arts", "medical", "dental"]
        selected_category = None

        for cat in categories:
            if cat in text:
                selected_category = cat
                break
```

```
    locations = ["kochi", "kozhikode", "thrissur", "kannur","trivandrum", "kollam",
"muvattupuzha"]
    selected_location = None

    for loc in locations:
        if loc in text:
            selected_location = loc
            break

    filtered = []

    for college in data["colleges"]:
        name_lower = college["name"].lower()

        if selected_category and college["category"] !=selected_category:
            continue

        if selected_location and selected_location not in name_lower:
            continue

        filtered.append(college)

    if filtered:
        top = sorted(filtered, key=lambda x: x["rating"], reverse=True)[:3]

        response = "Top colleges:\n"
        for c in top:
```

```
response += f'{c["name"]} (Rating: {c["rating"]})\n'
        else:
            response = "No matching colleges found."

    # ------------------ Colleges ------------------
    elif "college" in text:
        colleges = [c["name"] for c in data["colleges"]]
        response = "Here are some colleges:\n" + "\n".join(colleges[:10])

    # ------------------ Degree Courses ------------------
    elif any(word in text for word in ["degree", "b.tech", "bsc", "mba", "bca", "mbbs"]):
        courses = []
        for college in data["colleges"]:
            for course in college["courses"]:
                courses.append(f'{course["name"]} - {college["name"]}')

        response = "Degree courses available:\n" "\n".join(courses[:15])

    # ------------------ Hostel ------------------
    elif "hostel" in text:
        if last_course:
            response = f'Hostel info for {last_course["name"]}: {last_course.get("hostel",
"Not specified")}'
        else:
            response = "Most colleges provide hostel or nearby accommodation (₹3000–
6000/month)"

    # ------------------ Placement ------------------
```

```
elif "placement" in text or "place" in text:
    if last_course:
        companies = last_course.get("placement_companies", [])
        if companies:
            response = f'Placement support includes {", ".join(companies)}.'
        else:
            response = "Placement details not available."
    else:
        response = "Placement support varies by domain (IT, healthcare, finance, core industries)."

# ------------------ Fees ------------------
elif "fee" in text or "fees" in text:
    if last_course:
        response = f'The fees for {last_course["name"]} is {last_course["fees"]}.'
    else:
        response = "Please specify the course name to get fee details."

# ------------------ Short-term Courses ------------------
elif "course" in text:

    matched_courses = []

    for course in data["short_term_courses"]:
        name = course["name"].lower()
        category = course["category"].lower()
```

```
if name in text or category in text:
            matched_courses.append(course)

        if matched_courses:
            last_course = matched_courses[0]

            if len(matched_courses) == 1:
                c = matched_courses[0]
                response = (
                    f'{c["name"]} is a {c["duration"]} course with fees {c["fees"]}. '
                    f'Placement companies include {", ".join(c["placement_companies"])}.'
                )
            else:
                course_list = [f'{c["name"]} ({c["duration"]}, {c["fees"]})' for c in
matched_courses]
                response = "Matching courses:\n" + "\n".join(course_list)
        else:
            all_courses = [f'{c["name"]} ({c["duration"]}, {c["fees"]})' for c in
data["short_term_courses"]]
            response = "Available short-term courses:\n" + "\n".join(all_courses)

    # ------------------ Admission ------------------
    elif "admission" in text:
        response = "Admissions are open. Apply through official portals."

    # ------------------ Help ------------------
    elif "help" in text:
```

```
    response = "You can ask about colleges, courses, placements, hostel, or short-term courses."

    # ------------------ Default ------------------
    else:
        response = "Sorry, I didn't understand your query. Please try asking in a different way."

    logging.info(f"User: {user_input} | Bot: {response}")

    return response
```

- **dataset_generator.py**
  This script is used to generate structured data for colleges, courses, fees, and other details. It creates the dataset in JSON format.

```
import json
import random

engineering_colleges = [
"College of Engineering Trivandrum",
"Government Engineering College Thrissur",
"Government Engineering College Kozhikode",
"Model Engineering College Kochi",
"Rajagiri School of Engineering Kochi",
"TKM College of Engineering Kollam",
"Mar Athanasius College of Engineering Kothamangalam",
"Federal Institute of Science and Technology Kochi",
"LBS College of Engineering Kasaragod",
"SCMS School of Engineering Kochi",
```

```
"Amal Jyothi College of Engineering Kanjirappally",
"Saintgits College of Engineering Kottayam",
"Vidya Academy of Science and Technology Thrissur",
"Ilahia College of Engineering Muvattupuzha",
"MES College of Engineering Kuttippuram",
"Government Engineering College Kannur",
"Government Engineering College Idukki",
"College of Engineering Adoor",
"University College of Engineering Kariavattom",
"Government College of Engineering Barton Hill"
]

arts_colleges = [
"St Teresa’s College Kochi",
"Sacred Heart College Thevara",
"St Joseph’s College Devagiri",
"Farook College Kozhikode",
"Zamorin’s Guruvayurappan College Kozhikode",
"Mar Ivanios College Trivandrum",
"Christ College Irinjalakuda",
"Government College Thrissur",
"Brennen College Thalassery",
"Maharaja’s College Kochi",
"CMS College Kottayam",
"St Thomas College Thrissur",
"SN College Kollam",
"Government Victoria College Palakkad",
```

```
"Pazhassi Raja College Pulpally",
"Payyanur College Kannur",
"MMNSS College Kottiyam",
"St Albert's College Kochi",
"NSS College Ottapalam",
"SD College Alappuzha"
]

medical_colleges = [
"Government Medical College Thiruvananthapuram",
"Government Medical College Kozhikode",
"Government Medical College Kottayam",
"Government Medical College Thrissur",
"Government Medical College Kannur",
"Government Medical College Ernakulam",
"Government Medical College Manjeri",
"Government Medical College Kollam",
"Amrita Institute of Medical Sciences Kochi",
"Aster Medcity Kochi",
"Jubilee Mission Medical College Thrissur",
"Pushpagiri Medical College Thiruvalla",
"KMCT Medical College Kozhikode",
"Malabar Medical College Kozhikode",
"Sree Gokulam Medical College Trivandrum",
"Believers Church Medical College Thiruvalla",
"Dr Somervell Memorial CSI Medical College Karakonam",
"Travancore Medical College Kollam"
```

```
]

dental_colleges = [
"Government Dental College Thiruvananthapuram",
"Government Dental College Kozhikode",
"Amrita School of Dentistry Kochi",
"Pushpagiri College of Dental Sciences",
"KMCT Dental College Kozhikode",
"Royal Dental College Chalissery",
"Annoor Dental College Muvattupuzha",
"Indira Gandhi Institute of Dental Sciences Nellikuzhi",
"Educare Institute of Dental Sciences Malappuram",
"Malabar Dental College Edappal"
]

engineering_courses = ["B.Tech Computer Science", "B.Tech Mechanical", "B.Tech Civil", "B.Tech Electrical", "B.Tech Electronics"]
arts_courses = ["BCA", "BSc Computer Science", "BCom", "BBA", "BA English"]
medical_courses = ["MBBS", "BSc Nursing", "B.Pharm", "BPT", "BSc MLT"]
dental_courses = ["BDS", "MDS", "Dental Hygiene", "Dental Technology", "Oral Pathology"]

# COMPANY GROUPS
it_companies = ["TCS", "Infosys", "Wipro", "UST", "Cognizant"]
core_companies = ["L&T", "Tata Projects", "Bosch"]
finance_companies = ["HDFC Bank", "ICICI Bank", "EY", "Deloitte"]
healthcare_companies = ["Apollo", "Aster", "Sunrise Hospital", "Lakeshore Hospital"]
dental_companies = ["Dental Clinics", "Private Dental Hospitals"]
```

```
def get_companies(category):
    if category == "engineering":
        return random.sample(it_companies + core_companies, 3)
    elif category == "arts":
        return random.sample(it_companies + finance_companies, 3)
    elif category == "medical":
        return random.sample(healthcare_companies, 3)
    elif category == "dental":
        return random.sample(dental_companies, 2)
    else:
        return ["General Company"]

# COLLEGE CREATOR
def create_college(name, category):

    if category == "engineering":
        courses = engineering_courses
        base_fee = random.randint(40000, 90000)

    elif category == "arts":
        courses = arts_courses
        base_fee = random.randint(15000, 35000)

    elif category == "medical":
        courses = medical_courses
        base_fee = random.randint(100000, 500000)
```

```
else:
    courses = dental_courses
    base_fee = random.randint(150000, 400000)

course_data = []

for c in courses:
    course_data.append({
        "name": c,
        "fees": f"₹{base_fee + random.randint(-5000, 15000)}/year"
    })

return {
    "name": name,
    "category": category,
    "rating": round(random.uniform(3.8, 4.7), 1),
    "placement_support": {
        "available": True,
        "companies": get_companies(category),
        "training": "Aptitude + Technical + Interview prep"
    },
    "hostel": {
        "college_hostel": random.choice(["Available", "Not Available"]),
        "nearby_hostel": f"₹{random.randint(3000,6000)}/month"
    },
    "courses": course_data
}
```

```
# DATASET GENERATION
colleges = []

for c in engineering_colleges:
    colleges.append(create_college(c, "engineering"))

for c in arts_colleges:
    colleges.append(create_college(c, "arts"))

for c in medical_colleges:
    colleges.append(create_college(c, "medical"))

for c in dental_colleges:
    colleges.append(create_college(c, "dental"))

# SHORT TERM COURSES (UNCHANGED, JUST FIXED POSITION)
short_term_courses = [
    {
        "name": "Python Programming",
        "category": "IT",
        "duration": "3 months",
        "fees": "₹5000",
        "placement_companies": ["TCS", "Infosys", "Wipro", "UST"],
        "hostel": "Available near institute (₹3000/month)"
    },
    {
        "name": "Full Stack Development",
```

```
  "category": "IT",
        "duration": "6 months",
        "fees": "₹18000",
        "placement_companies": ["Zoho", "Freshworks", "Infosys", "TCS"],
        "hostel": "Available (₹4000/month)"
    }
]

# FINAL DATASET
dataset = {
    "colleges": colleges,
    "short_term_courses": short_term_courses
}

# SAVE
with open("data/dataset.json", "w") as f:
    json.dump(dataset, f, indent=2)

print("Dataset generated successfully!")
```

- **index.html**
  This file defines the frontend user interface of the chatbot. It includes the chat layout, input field, and communication with the backend using JavaScript.

```
<!DOCTYPE html>
<html>
<head>
    <title>EdTech Chatbot</title>
    <style>
        body {
```

```
margin: 0;
    font-family: Arial, sans-serif;
    background: #ece5dd;
}

#container {
    width: 100%;
    max-width: 500px;
    margin: auto;
    height: 100vh;
    display: flex;
    flex-direction: column;
    border: 1px solid #ccc;
}

#header {
    background: #075e54;
    color: white;
    padding: 15px;
    text-align: center;
    font-size: 18px;
}

#messages {
    flex: 1;
    padding: 10px;
    overflow-y: auto;
```

```
    background: #e5ddd5;
  }

  .message {
    margin: 8px 0;
    max-width: 75%;
    padding: 10px;
    border-radius: 10px;
    word-wrap: break-word;
  }

  .user {
    background: #dcf8c6;
    margin-left: auto;
    text-align: right;
  }

  .bot {
    background: white;
    margin-right: auto;
    text-align: left;
  }

  #inputArea {
    display: flex;
    padding: 10px;
    background: #f0f0f0;
```

```
    }

        input {
            flex: 1;
            padding: 10px;
            border-radius: 20px;
            border: 1px solid #ccc;
            outline: none;
        }

        button {
            margin-left: 10px;
            padding: 10px 15px;
            border: none;
            background: #075e54;
            color: white;
            border-radius: 50%;
            cursor: pointer;
        }

        button:hover {
            background: #0b7a6e;
        }
    </style>
</head>
<body>
```

```
<div id="container">
    <div id="header">EdTech Chatbot</div>

    <div id="messages"></div>

    <div id="inputArea">
        <input type="text" id="userInput" placeholder="Type a message..."
onkeypress="handleKey(event)">
        <button onclick="sendMessage()">➤</button>
    </div>
</div>

<script>
function sendMessage() {
    let input = document.getElementById("userInput");
    let message = input.value;

    if (message.trim() === "") return;

    let messages = document.getElementById("messages");

    messages.innerHTML += `<div class='message user'>${message}</div>`;

    fetch("/chat", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
```

```
    },
        body: JSON.stringify({ message: message })
    })
    .then(response => response.json())
    .then(data => {
        messages.innerHTML += `<div class='message bot'>${data.response}</div>`;
        messages.scrollTop = messages.scrollHeight;
    });

    input.value = "";
}

function handleKey(event) {
    if (event.key === "Enter") {
        sendMessage();
    }
}
</script>

</body>
</html>
```

8.2 Dataset Description

The chatbot uses a JSON dataset to store structured information about colleges and courses. The dataset includes:

- College name and category
- Ratings
- Course details and fees

- Placement support information
- Hostel availability

The dataset is designed in a hierarchical format, making it easy to retrieve relevant information during query processing.

8.3 Sample Execution Logs

2026-04-15 22:12:05,628 - [31m[1mWARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.[0m

* Running on http://127.0.0.1:5000

2026-04-15 22:12:05,629 - [33mPress CTRL+C to quit[0m

2026-04-15 22:13:19,237 - 127.0.0.1 - - [15/Apr/2026 22:13:19] "GET / HTTP/1.1" 200 -

2026-04-15 22:13:19,685 - 127.0.0.1 - - [15/Apr/2026 22:13:19] "[33mGET /favicon.ico HTTP/1.1[0m" 404 -

2026-04-15 22:13:19,939 - 127.0.0.1 - - [15/Apr/2026 22:13:19] "GET / HTTP/1.1" 200 -

2026-04-15 22:13:24,681 - User: hi | Bot: Hello! How can I assist you with courses or colleges today?

2026-04-15 22:13:24,681 - 127.0.0.1 - - [15/Apr/2026 22:13:24] "POST /chat HTTP/1.1" 200 -

2026-04-15 22:16:16,905 - User: courses | Bot: Available short-term courses:

Python Programming (3 months, \u20b95000)

Full Stack Development (6 months, \u20b918000)

2026-04-15 22:16:16,921 - 127.0.0.1 - - [15/Apr/2026 22:16:16] "POST /chat HTTP/1.1" 200 -

2026-04-15 22:16:29,411 - User: fees | Bot: Please specify the course name to get fee details.

2026-04-15 22:16:29,411 - 127.0.0.1 - - [15/Apr/2026 22:16:29] "POST /chat HTTP/1.1" 200 -

2026-04-15 22:16:52,626 - User: admission | Bot: Admissions are open. Apply through official portals.

2026-04-15 22:16:52,626 - 127.0.0.1 - - [15/Apr/2026 22:16:52] "POST /chat HTTP/1.1" 200 -

2026-04-15 22:18:31,606 - 127.0.0.1 - - [15/Apr/2026 22:18:31] "GET / HTTP/1.1" 200 -

2026-04-15 22:18:57,978 - [31m[1mWARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.[0m

* Running on http://127.0.0.1:5000

The system maintains logs of user interactions for monitoring and debugging purposes. These logs include user queries and corresponding chatbot responses.

Example entries include:

- User queries related to courses and colleges
- Chatbot responses based on dataset matching
- System execution details

These logs help in analyzing system performance and identifying issues.

8.4 System Execution Steps

The following steps are used to run the chatbot system:

1. Install Python and required libraries
2. Place all project files in the appropriate folder structure
3. Run the Flask application using app.py
4. Open a web browser and navigate to: **http://127.0.0.1:5000/**
5. Enter queries and interact with the chatbot